

Introduction to Social Data Analytics: Week 10 Part 1

Today: Functions in R

By the end of today's lecture, you should be able to:

- ▶ Describe this week's empirical application: unconditional cash transfers
- ▶ Describe how functions can save time and space while writing code
- ▶ Construct functions that perform basic calculations
- ▶ Identify inputs and outputs of a function

This Week's Application: Unconditional Cash Transfers

- ▶ Haushofer and Shapiro (2016) studies the impact of unconditional cash transfers in Kenya on the economic and psychological well-being of recipients
 - ▶ Unconditional means households can spend the cash however they want
 - ▶ This differentiates them from conditional cash transfers, which can only be used on certain things
- ▶ They study this question by randomizing unconditional cash transfers across villages and households

Give Directly

- ▶ The study was possible due to the charitable organization, Give Directly, a charity co-founded by Paul Niehaus, professor of economics at UCSD
 - ▶ Main goal of the charity is to alleviate poverty by giving funds directly to individuals in poverty (i.e. Give Directly)
- ▶ Currently running a universal basic income experiment (UBI) in a rural region in Kenya
 - ▶ This is a massive program, providing UBI to many villages for 12 years
 - ▶ The hope is that the results can help inform similar UBI experiments/policies in the future

The Treatment

- ▶ This paper randomizes (1) whether a household receives an unconditional cash transfer, (2) the timing of the transfer (monthly vs. lump sum) and (3) the size of the transfer (small \$400 vs large \$1500)
- ▶ For our purposes we will be considering differences between households that received any transfer vs households that received no transfer

Outcomes

- ▶ We could use this data to study the impact of these unconditional cash transfers on a larger number of outcomes
 - ▶ Assets
 - ▶ Expenditures
 - ▶ Revenue
 - ▶ Food security
 - ▶ Health index
 - ▶ Education index
 - ▶ Psychological index
- ▶ We will focus on a subset of these

Functions

- ▶ In R, we have already been using a lot of functions
 - ▶ Functions are anything that take in an input and produce some output
 - ▶ For example, the `mean()` function takes in a list of numbers and outputs the average of that list of numbers
- ▶ In R, we can actually create our own functions

Writing a Function

- ▶ To write a function you need to specify a few things
 - ▶ **name**: what is the name of the function? You get to choose!
 - ▶ **body**: what does the function do?
 - ▶ **input**: what does the function take in as input
 - ▶ **output**: what does the function produce as an output

Basic syntax

```
functionname <- function(input) {  
  output <- Code that performs some operation on input  
  return(output)  
}
```

- ▶ `functionname` is chosen by you. That's the name of the function
- ▶ `return(output)` tells R what to return to the user

Example (the mean function)

- ▶ Let's write a new function `mymean` that takes the mean of a vector

```
mymean <- function(num) {  
  
  output <- sum(num)/length(num)  
  return(output)  
  
}
```

- ▶ `num` is a vector of numbers. Now whenever I type `mymean(num)`, R will know to take the average of that list of numbers

mymean vs. mean

- If we use mymean to take average of `c(3,5,7)` we get

```
myvec <- c(3,5,7)
```

```
mymean(myvec)
```

```
## [1] 5
```

- We can check that we got the right output by using the built-in `mean()` function

```
mean(myvec)
```

```
## [1] 5
```

Load the data

```
library(tidyverse)
uct <- read_csv("uct.csv")
head(uct)
```

```
## # A tibble: 6 x 19
```

```
##   treat village treatX~1 asset~2 asset~3 nondu~4 nondu~5 total~6 total~7
##   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1     0     74     0    240.    128.    163.    56.6    21.1    2.43
## 2     1     55     0    665.    395.    201.    156.    102.    21.7
## 3     0     71     0   1384.    731.    93.2    162.    59.3   115.
## 4     0     46     0    201.    214.    56.1    133.    244.   533.
## 5     1     15     0    189.    271.    65.8     6.28    6.01    0.40
## 6     0     61     0    387.    229.    180.    237.    21.0   92.0
## # ... with 9 more variables: food_security_before <dbl>,
## #   health_index_after <dbl>, health_index_before <dbl>, ed_index_after
## #   ed_index_before <dbl>, psy_index_after <dbl>, psy_index_before <dbl>,
## #   treatXlarge <dbl>, treatXmonthlyXsmall <dbl>, and abbreviated variables
```

Calculate mean assets after experiment for treated and control

```
mean(uct$asset_after[uct$treat == 1])
```

```
## [1] 790.9253
```

```
mean(uct$asset_after[uct$treat == 0])
```

```
## [1] 494.8041
```

Using tidyverse

```
#For treated  
uct %>%  
  filter(treat==1) %>%  
  summarize(meanassetafter = mean(asset_after))
```

```
## # A tibble: 1 x 1  
##   meanassetafter  
##           <dbl>  
## 1           791.
```

Using tidyverse

```
#For control
```

```
uct %>%
```

```
  filter(treat==0) %>%
```

```
  summarize(meanassetafter = mean(asset_after))
```

```
## # A tibble: 1 x 1
```

```
##   meanassetafter
```

```
##           <dbl>
```

```
## 1           495.
```

Calculate mean asset after experiment for treated and control

```
treat.mean <- function(treatvalue){  
  return(mean(uct$asset_after[uct$treat==treatvalue]))  
}
```

```
treat.mean(0)
```

```
## [1] 494.8041
```

```
treat.mean(1)
```

```
## [1] 790.9253
```

Calculate mean assets after experiment for control

```
treat.mean <- function(treatvalue){  
  output <- uct %>%  
    filter(treat==treatvalue) %>%  
    summarize(meanasset = mean(asset_after))  
  return(output)  
}
```

```
treat.mean(0)
```

```
## # A tibble: 1 x 1  
##   meanasset  
##   <dbl>  
## 1      495.
```

Calculate mean asset after experiment for treated

```
treat.mean <- function(treatvalue){  
  output <- uct %>%  
    filter(treat==treatvalue) %>%  
    summarize(meanasset = mean(asset_after))  
  return(output)  
}
```

```
treat.mean(1)
```

```
## # A tibble: 1 x 1  
##   meanasset  
##   <dbl>  
## 1      791.
```

What Next?

- ▶ So far the inputs have been numbers
- ▶ Next we are going to discuss how to use strings in functions
- ▶ Our goal: for each of these variables: assets, total revenue, nondurable expenditures
 - ▶ 1. Compute averages of treated vs. control before experiment
 - ▶ 2. Compute averages of treated vs. control after experiment
- ▶ To create this function we will need to discuss how to use strings in functions in R

Setup

- ▶ To figure out how to create our function, we will start with a single variable and then generalize
- ▶ Let's discuss how to compute average assets before and after for the asset variable
- ▶ To start, let's create an object named variable that is equal to asset

```
variable <- "asset"
```

The paste() function

- ▶ We want to construct two names from “asset”: “asset_after” and “asset_before”
- ▶ To concatenate two string we can use the paste() function

```
aftervar <- paste(variable, "_after", sep="")  
beforevar <- paste(variable, "_before", sep="")
```

- ▶ The option sep="" tells R not to include a space between “asset” and “_after”
 - ▶ We want no space because in our data frame the variable name is “asset_after”

Pull() function

- ▶ We are now going to create a treated dataframe and control dataframe
- ▶ The `pull()` function grabs columns of your data frame
- ▶ We will use it to extract certain columns of our new dataframes

Why pull()

- ▶ Imagine we want to take mean of `asset_after` in `uct_treat`
 - ▶ Easy: just type `mean(uct_treat$asset_after)`
- ▶ However, now we want to use the values we have created `aftervar` and `beforevar`
- ▶ To do this we can use the `pull()` function

pull()

- ▶ The code:

```
pull(uct_treat, aftervar)
```

- ▶ Will look into the dataframe `uct_treat` and pull out `aftervar` which is equal to `asset_after`
- ▶ This is equivalent to `uct_treat$asset_after`. The reason we are doing it this way is that we eventually want to do this within a function in which we change the name of the variable each time we use the function

Using pull to compute averages

```
#Treated After
```

```
mean(pull(uct_treat, aftervar))
```

```
## [1] 790.9253
```

```
#Treated Before
```

```
mean(pull(uct_treat, beforevar))
```

```
## [1] 382.3934
```

```
#Control After
```

```
mean(pull(uct_control, aftervar))
```

```
## [1] 494.8041
```

```
#Control Before
```

```
mean(pull(uct_control, beforevar))
```

```
## [1] 389.371
```

Making it General

- ▶ Now that we have figured out how to do this with a single variable we can generalize it to a function
- ▶ Then we can compute the averages before and after for treated and control workers for many different variables quickly

Let's make this general with a function

```
treat.mean.var <- function(variable){  
  aftervar <- paste(variable, "_after", sep="")  
  beforevar <- paste(variable, "_before", sep="")  
  
  #Created treated and control datasets  
  uct_treat <- filter(uct, treat==1)  
  uct_control <- filter(uct, treat==0)  
  
  #Treated After  
  treatedafter <- mean(pull(uct_treat, aftervar))  
  #Treated Before  
  treatedbefore <- mean(pull(uct_treat, beforevar))  
  
  #Control After  
  controlafter <- mean(pull(uct_control, aftervar))  
  #Control Before  
  controlbefore <- mean(pull(uct_control, beforevar))  
  
  output <- c(treatedafter, treatedbefore, controlafter, controlbefore)  
  names(output) <- c("Treated After", "Treated Before",  
    "Control After", "Control Before")  
  return(output)  
}
```

Now we can call the function

```
treat.mean.var("asset")
```

```
##   Treated After Treated Before Control After Control Before
##           790.9253           382.3934           494.8041           389.3710
```

```
treat.mean.var("nondurable_expend")
```

```
##   Treated After Treated Before Control After Control Before
##           191.2080           174.3455           157.6120           183.5800
```

```
treat.mean.var("total_rev")
```

```
##   Treated After Treated Before Control After Control Before
##           62.03139           52.93187           48.98467           88.88401
```